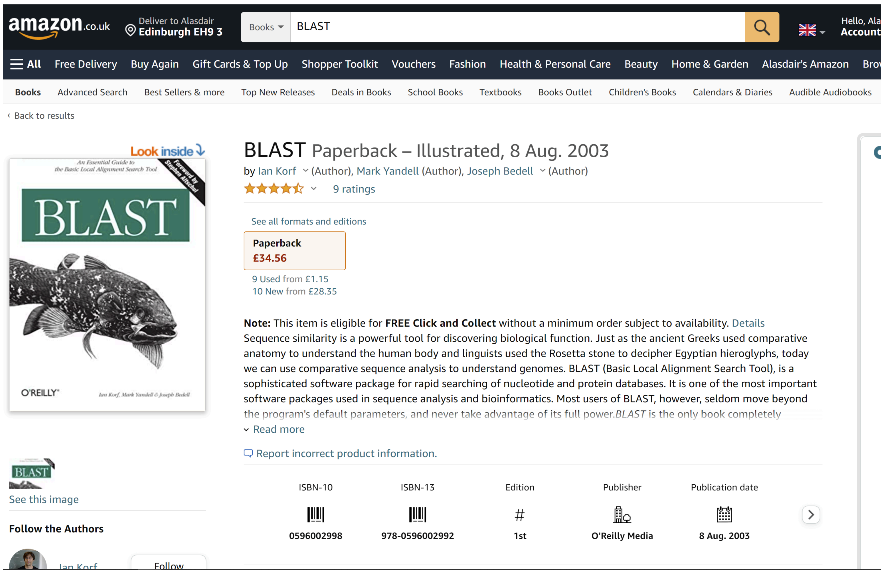
Basic Local Alignment Search Tool
https://bioinfmsc8.bio.ed.ac.uk/AY25_BPSM06.html

Quick links:
Click on any to get back here!

Basic Local Alignment Search Tool

Basic **L**ocal **A**lignment **S**earch **T**ool
Altschul SF, Gish W, Miller W, Myers EW, Lipman DJ
J Mol Biol. 1990 215:403-410.
http://www.sciencedirect.com/science/article/pii/S0022283605803602
http://www.ncbi.nlm.nih.gov/pubmed/2231712

Other information sources
http://blast.ncbi.nlm.nih.gov/Blast.cgi
https://www.youtube.com/watch?v=WRKQGwh_Mw0
https://guides.lib.berkeley.edu/ncbi/blast http://en.wikipedia.org/wiki/BLAST



Available from Amazon (but it's ~~quite~~ very old now...!)
https://www.amazon.co.uk/BLAST-Ian-Korf/dp/0596002998

What does BLAST do?

It's a programme that will identify putatively homologous sequences in a database

- statistically likely, but not certain
- related by evolution
- DNA and protein
- a collection of existing sequences

What problem does BLAST solve?

Pairwise alignment tools (e.g. Smith-Waterman) are guaranteed to find the best alignment but are very slow (especially on large databases!)

BLAST is not guaranteed to find the best alignment, but is many orders of magnitude quicker

What other things can BLAST be used for?

- functional identification of novel sequences
- annotation of genomes
- localisation of particular sequences in genomes
- screening of environments using set of sequences
- clustering of sequence data to identify sequence families
- … and many, many more

Magic blast (for paired end RNAseq data)!

https://ncbi.github.io/magicblast

NCBI Magic-BLAST (/magicblast/) RNA-seq mapping tool

Home (/magicblast/)

DOCUMENTATION

How Magic-BLAST works (/magicblast/doc/tutorial.html)
Output (/magicblast/doc/output.html)
Download (/magicblast/doc/download.html)
Feedback (/magicblast/doc/feedback.html)

COOKBOOK

Create a BLAST database (/magicblast/cook/blastdb.html)
Use NCBI SRA repository (/magicblast/cook/sra.html)
Reads in FASTA or FASTQ (/magicblast/cook/fasta.html)
Paired reads (/magicblast/cook/paired.html)
RNA vs DNA (/magicblast/cook/rnavsdna.html)
Multi-threading (/magicblast/cook/multithreading.html)
Access SRA reads in the cloud (/magicblast/cook/cloud-sra.html)

DEVELOPERS

License (/magicblast/dev/license.html)
Copyright (/magicblast/dev/copyright.html)
Exceptions (/magicblast/dev/exeptions.html)

RELEASE NOTES

And ElasticBLAST for in the cloud

IMPORTANT: ElasticBLAST version prior to 1.4.0 may fail to properly release cloud resources, which could lead to excess user charges. We recommend upgrading ElasticBLAST to its latest version.
As a temporary workaround, we recommend using our AWS implementation or updating the required kubernetes version as outlined on the known issues page.

(Message date: March 17, 2025)

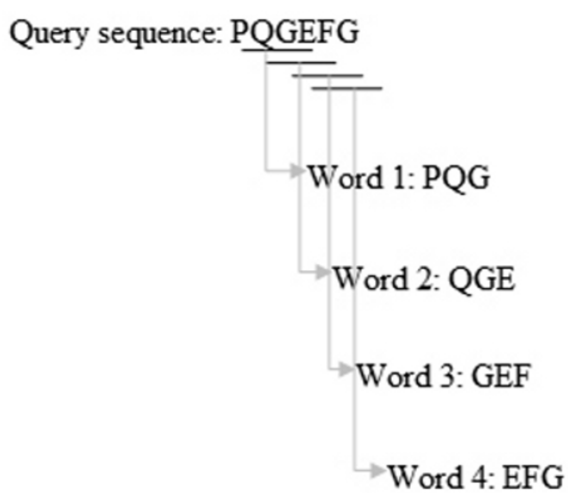
Notice

Because of a lapse in government funding, the information on this website may not be up to date, transactions submitted via the website may not be processed, and the agency may not be able to respond to inquiries until appropriations are enacted. The NIH Clinical Center (the research hospital of NIH) is open. For more details about its operating status, please visit cc.nih.gov. Updates regarding government operating status and resumption of normal operations can be found at opm.gov.

How does BLAST work?

Amusing video (11:39 mins long): https://www.youtube.com/watch?v=NwiETShpIDo

1. Builds an index of short words for all sequences in a database
2. Looks for matching short words in the query sequence (what you are searching with)
3. For each match (seed), extend it to the left and right to make the longest match (High scoring Segment Pair)
4. Stops extending it when the score stops going up
5. Try to merge HSPs to get even longer matches



How does BLAST work: a series of excellent short videos
Rob Edwards

1. Intro
2. Seed and extend
3. Amino acid differences
4. PAM vs BLOSUM
5. BLOSUM
6. Scoring alignments
7. Scoring gaps
8. Probability
9. E values
10. Algorithms
11. PSWM (position specific weight matrices)

How well do these sequences match?

Simple answer:
Use a scoring matrix to figure out the score for each position

	A	G	C	T	-
A	1	0	0	0	
G	0	1	0	0	
C	0	0	1	0	
T	0	0	0	1	
-					

How do we figure out the score of a match?

Trickier answer:
Use a scoring matrix to figure out the score for each position, then add them all up

Matrix

	A	G	C	T	-
A	1	0	0	0	
G	0	1	0	0	
C	0	0	1	0	
T	0	0	0	1	
-					

Alignments

```
GCTGCTACG
| || ||
G-TG-TAAA

101101100 = 5

GCTGCTACG
|||| || |
GCTG-TAAG

111101101 = 7
```

What about protein alignments?!

Amino acid scoring matrices are more interesting/challenging...

- Simple identity (like for DNA)
- Distance in genetic code (i.e. number of nucleotide substitutions required)
- Look at similarity of chemical properties
- Use existing alignments (e.g structural) and calculate frequencies of substitution

Something to think about:
if every amino acid is equally likely to occur in a protein sequence (which isn't true!), can you work out the probability of getting this 7 amino acids long sequence occurring in a protein?

`VCPKKCS`

What would this be if the "normal" frequency of cysteine residues was 0.01?!

What would this be if the "normal" frequency of cysteine-proline adjacent pairs was 0.00000001?!

What are the chances of me winning the Euromillions lottery??

PAM matrices

"A **p**oint **a**ccepted **m**utation (**PAM**) is the replacement of a single amino acid in the primary structure of a protein with another single amino acid, which is accepted by the processes of natural selection. "

PAM N corresponds to N mutations in DNA sequence per 100 amino acids. N can be > 100 and PAM250 is one of the more common used scoring matrices.

PAM model naming convention: high number for distantly related proteins, low number for closely related protein

C Cys	12																			
S Ser	0	2																		
T Thr	−2	1	3																	
P Pro	−3	1	0	6																
A Ala	−2	1	1	1	2															
G Gly	−3	1	0	−1	1	5														
N Asn	−4	1	0	−1	0	0	2													
D Asp	−5	0	0	−1	0	1	2	4												
E Glu	−5	0	0	−1	0	0	1	3	4											
Q Gln	−5	−1	−1	0	0	−1	1	2	2	4										
H His	−3	−1	−1	0	−1	−2	2	1	1	3	6									
R Arg	−4	0	−1	0	−2	−3	0	−1	−1	1	2	6								
K Lys	−5	0	0	−1	−1	−2	1	0	0	1	0	3	5							
M Met	−5	−2	−1	−2	−1	−3	−2	−3	−2	−1	−2	0	0	6						
I Ile	−2	−1	0	−2	−1	−3	−2	−2	−2	−2	−2	−2	−2	2	5					
L Leu	−6	−3	−2	−3	−2	−4	−3	−4	−3	−2	−2	−3	−3	4	2	6				
V Val	−2	−1	0	−1	0	−1	−2	−2	−2	−2	−2	−2	−2	2	4	2	4			
F Phe	−4	−3	−3	−5	−4	−5	−4	−6	−5	−5	−2	−4	−5	0	1	2	−1	9		
Y Tyr	0	−3	−3	−5	−3	−5	−2	−4	−4	−4	0	−4	−4	−2	−1	−1	−2	7	10	
W Trp	−8	−2	−5	−6	−6	−7	−4	−7	−7	−5	−3	2	−3	−4	−5	−2	−6	0	0	17
	C Cys	S Ser	T Thr	P Pro	A Ala	G Gly	N Asn	D Asp	E Glu	Q Gln	H His	R Arg	K Lys	M Met	I Ile	L Leu	V Val	F Phe	Y Tyr	W Trp

The PAM-250 Log Odds Substitution Matrix

BLOSUM matrices

The BLOSUM (**BLO**cks **SU**bstitution **M**atrix) matrix is a substitution matrix used for scoring sequence alignment of evolutionarily divergent protein sequences.

BLOSUM matrices are based on local alignments unlike PAM which is based on global (end to end) alignments of highly similar proteins.
BLOSUM matrices were first introduced in a paper by Henikoff and Henikoff.

1. They scanned the BLOCKS database for very conserved regions of protein families (UNGAPPED local alignments) and then counted the relative frequencies of amino acids and their substitution probabilities.
2. Then derived a log-odds score for each of the 210 possible substitution pairs of the 20 standard amino acids.
3. All BLOSUM matrices are based on observed alignments; they are not extrapolated from comparisons of closely related proteins like the PAM Matrices.
4. BLOSUM62 is the matrix built using sequences with no more than 62% similarity; BLOSUM62 is the default matrix used for protein BLAST.
5. BLOSUM80 is for less divergent alignments, whereas BLOSUM 50 is for more divergent alignments.

Give feedback
Ask for help

Example of a BLOSUM matrix

Ala	4																			
Arg	-1	5																		
Asn	-2	0	6																	
Asp	-2	-2	1	6																
Cys	0	-3	-3	-3	9															
Gln	-1	1	0	0	-3	5														
Glu	-1	0	0	2	-4	2	5													
Gly	0	-2	0	-1	-3	-2	-2	6												
His	-2	0	1	-1	-3	0	0	-2	8											
Ile	-1	-3	-3	-3	-1	-3	-3	-4	-3	4										
Leu	-1	-2	-3	-4	-1	-2	-3	-4	-3	2	4									
Lys	-1	2	0	-1	-3	1	1	-2	-1	-3	-2	5								
Met	-1	-1	-2	-3	-1	0	-2	-3	-2	1	2	-1	5							
Phe	-2	-3	-3	-3	-2	-3	-3	-3	-1	0	0	-3	0	6						
Pro	-1	-2	-2	-1	-3	-1	-1	-2	-2	-3	-3	-1	-2	-4	7					
Ser	1	-1	1	0	-1	0	0	0	-1	-2	-2	0	-1	-2	-1	4				
Thr	0	-1	0	-1	-1	-1	-1	-2	-2	-1	-1	-1	-1	-2	-1	1	5			
Trp	-3	-3	-4	-4	-2	-2	-3	-2	-2	-3	-2	-3	-1	1	-4	-3	-2	11		
Tyr	-2	-2	-2	-3	-2	-1	-2	-3	2	-1	-1	-2	-1	3	-3	-2	-2	2	7	
Val	0	-3	-3	-3	-1	-2	-2	-3	-3	3	1	-2	1	-1	-2	-2	0	-3	-1	4
	Ala	**Arg**	**Asn**	**Asp**	**Cys**	**Gln**	**Glu**	**Gly**	**His**	**Ile**	**Leu**	**Lys**	**Met**	**Phe**	**Pro**	**Ser**	**Thr**	**Trp**	**Tyr**	**Val**

PAM/BLOSUM: how do they really differ? A good short history/summary available here!

How is the "bitscore" of a match calculated?

The bitscore obtained depends on several parameters, including:

- the matrix used (e.g. BLOSUM62)
- the length of the query
- the word size
- the size of the database

Karlin-Altschul statistics are used to convert the HSP bitscore to an expectation value (E-value)

In practical terms:

- bigger HSP bitscores are "better" (866 is "better" than 155)
- smaller E-values are "better" (1.6e-12 is "better" than 5.3e-03)
- but not all HSPs are equally comparable, so use with care!

Guidance on selecting the right similarity-scoring matrix...

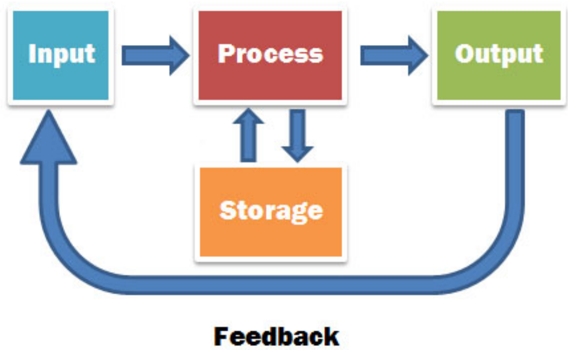
(Main) Flavours of BLAST

		Query	
		DNA	Protein
Database	DNA	blastn megablast tblastx magic_blast	tblastn
	Protein	blastx	blastp/quickblastp psi-blast

https://blast.ncbi.nlm.nih.gov/Blast.cgi?

Notice

Because of a lapse in government funding, the information on this website may not be up to date, transactions submitted via the website may not be processed, and the agency may not be able to respond to inquiries until appropriations are enacted. The NIH Clinical Center (the research hospital of NIH) is open. For more details about its operating status, please visit cc.nih.gov. Updates regarding government operating status and resumption of normal operations can be found at opm.gov.



Using BLAST

1. **Step 1: something to search against**
   - existing DNA/protein sequences
   - `makeblastdb`
   - BLAST database
2. **Step 2: do the search**
   - take existing query DNA/protein sequences
   - run `blastn` or `blastx` or `tblastn` or .... against database
3. **Step 3: process results**
   - inspect and process results
4. **Step 4 onwards: intepret the results**
   - hours, days, weeks....

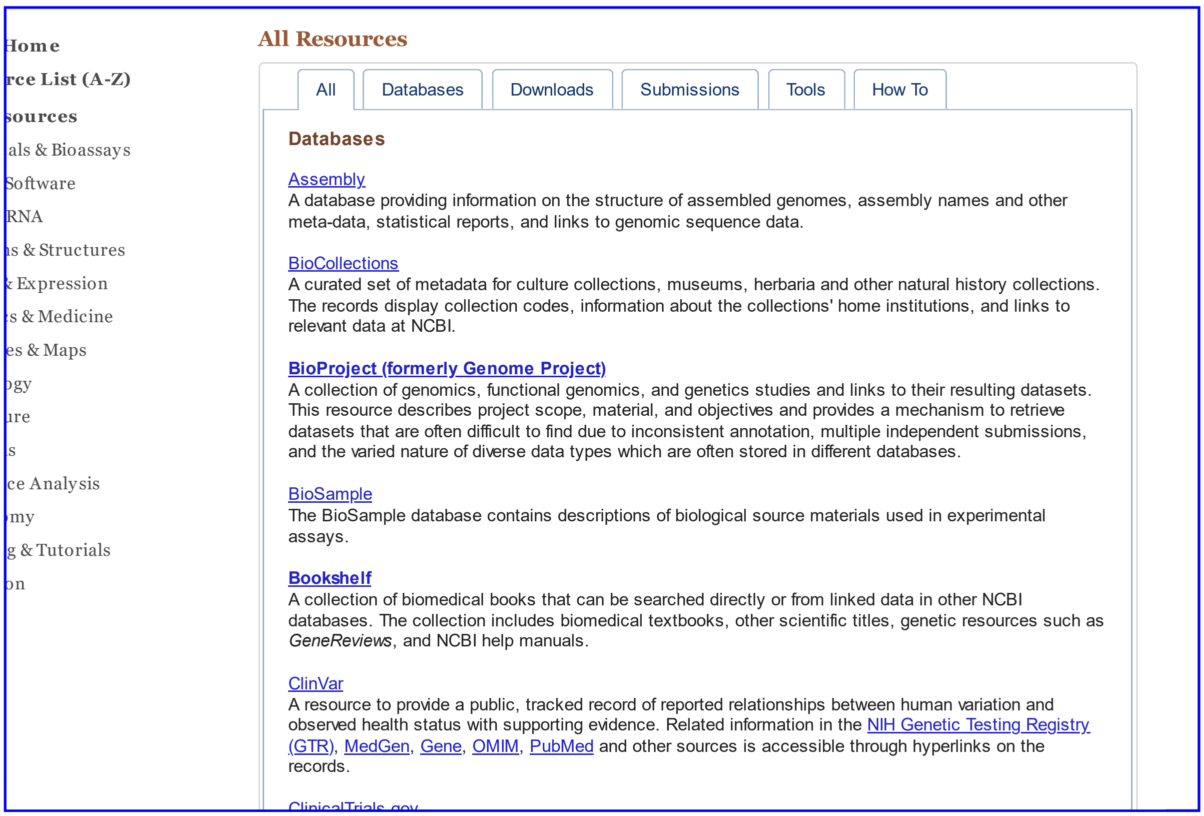
NCBI has a series of help videos for everything here, which includes:

- A practical guide to NCBI BLAST (old (2016), but still useful? 54mins)

- A practical guide to NCBI BLAST ON THE WEB (old (2015), but still useful? 1h22mins)
- BLAST results: Expect values #1
- BLAST results: Expect values #2

Which database(s) to use?

There are a huge number of resources available, e.g. at NCBI https://www.ncbi.nlm.nih.gov/guide/all/ which houses a vast array of different datasets.



For sequence analyses, we *could* download one of more of the following databases:

- nr : non-redundant protein database
- nt : non-redundant nucleotide database
- swiss-prot : manually annotated protein database
- pat : protein sequences from patents
- month : recently added sequences

- env_nr : environmental sequences
- est_human : only human Expressed Sequence Tag sequences
- and many more....

but generally, it is better to **access them remotely**, if possible: the databases are HUGE, and frequently updated (potential versioning issues)!

...or if we have a small dataset, we could just build our own database

To illustrate this, we are going to obtain a small collection of nematode sequences from a remote file:

Warning: do **NOT** try downloading these sequences into your browser...

BAD THINGS WILL HAPPEN!!

https://live.bio.ed.ac.uk/bioinfmsc/nem.fasta

First, we need to get the file of sequences onto our local server...

Action plan for today

1. Log in to bioinfmsc7 or bioinfmsc8 (**not** bioinfmsc9)...
2. ...and use git...
3. ...to version control all your bash and awk scripts...
4. ...written using Linux/awk commands...
5. ...in your favourite plain text editor, for example...
   - vi (vi cheatsheet)
   - nano (nano cheatsheet)
6. ...maybe sometimes with a bit of other bash help?

Let's try and be tidy...(think git)

 # Getting ready for BLAST
cd
ls -al
....some files....

Previous command shows us the files in our home directory, with all the permissions, dates, etc.: what we did last time should still be there. Can you remember what all the files were for?!

 # Setting up a working directory for today
mkdir ~/Exercises/Lecture06
cd ~/Exercises/Lecture06
ls -al

```
total 4
drwxrwxr-x 2 s0000000 users    0 Oct  3 10:36 .
drwxrwxr-x 3 s0000000 users 4096 Oct  3 10:36 ..
```

Get the sequence

Check where we are

pwd

/home/s0000000/Exercises/Lecture06/

`wget` is the name of the programme we will use to get the remote file

How do I use wget?

wget --help

```
GNU Wget 1.21.4, a non-interactive network retriever.
Usage: wget [OPTION]... [URL]...

Mandatory arguments to long options are mandatory for short options too.

Startup:
  -V,  --version                   display the version of Wget and exit
  -h,  --help                      print this help
  -b,  --background                go to background after startup
  -e,  --execute=COMMAND           execute a `.wgetrc'-style command

Logging and input file:
  -o,  --output-file=FILE          log messages to FILE
  -a,  --append-output=FILE        append messages to FILE
  -d,  --debug                     print lots of debugging information
  -q,  --quiet                     quiet (no output)
  -v,  --verbose                   be verbose (this is the default)
  -nv, --no-verbose                turn off verboseness, without being quiet
       --report-speed=TYPE         output bandwidth as TYPE.  TYPE can be bits
  -i,  --input-file=FILE           download URLs found in local or external FILE
  -F,  --force-html                treat input file as HTML
  -B,  --base=URL                  resolves HTML input-file links (-i -F)
                                     relative to URL
       --config=FILE               specify config file to use
       --no-config                 do not read any config file
       --rejected-log=FILE         log reasons for URL rejection to FILE

Download:
  -t,  --tries=NUMBER              set number of retries to NUMBER (0 unlimits)
       --retry-connrefused         retry even if connection is refused
       --retry-on-host-error       consider host errors as non-fatal, transient errors
       --retry-on-http-error=ERRORS    comma-separated list of HTTP errors to retry
  -O,  --output-document=FILE      write documents to FILE
  -nc, --no-clobber                skip downloads that would download to
                                     existing files (overwriting them)
       --no-netrc                  don't try to obtain credentials from .netrc
  -c,  --continue                  resume getting a partially-downloaded file
       --start-pos=OFFSET          start downloading from zero-based position OFFSET
       --progress=TYPE             select progress gauge type
       --show-progress             display the progress bar in any verbosity mode
  -N,  --timestamping              don't re-retrieve files unless newer than
                                     local
       --no-if-modified-since      don't use conditional if-modified-since get
                                     requests in timestamping mode
       --no-use-server-timestamps  don't set the local file's timestamp by
                                     the one on the server
  -S,  --server-response           print server response
       --spider                    don't download anything
  -T,  --timeout=SECONDS           set all timeout values to SECONDS
       --dns-timeout=SECS          set the DNS lookup timeout to SECS
       --connect-timeout=SECS      set the connect timeout to SECS
       --read-timeout=SECS         set the read timeout to SECS
  -w,  --wait=SECONDS              wait SECONDS between retrievals
                                     (applies if more then 1 URL is to be retrieved)
       --waitretry=SECONDS         wait 1..SECONDS between retries of a retrieval
                                     (applies if more then 1 URL is to be retrieved)
       --random-wait               wait from 0.5*WAIT...1.5*WAIT secs between retrievals
                                     (applies if more then 1 URL is to be retrieved)
       --no-proxy                  explicitly turn off proxy
  -Q,  --quota=NUMBER              set retrieval quota to NUMBER
       --bind-address=ADDRESS      bind to ADDRESS (hostname or IP) on local host
```

```
       --limit-rate=RATE           limit download rate to RATE
       --no-dns-cache              disable caching DNS lookups
       --restrict-file-names=OS    restrict chars in file names to ones OS allows
       --ignore-case               ignore case when matching files/directories
  -4,  --inet4-only                connect only to IPv4 addresses
  -6,  --inet6-only                connect only to IPv6 addresses
       --prefer-family=FAMILY      connect first to addresses of specified family,
                                     one of IPv6, IPv4, or none
       --user=USER                 set both ftp and http user to USER
       --password=PASS             set both ftp and http password to PASS
       --ask-password              prompt for passwords
       --use-askpass=COMMAND       specify credential handler for requesting
                                     username and password.  If no COMMAND is
                                     specified the WGET_ASKPASS or the SSH_ASKPASS
                                     environment variable is used.
       --no-iri                    turn off IRI support
       --local-encoding=ENC        use ENC as the local encoding for IRIs
       --remote-encoding=ENC       use ENC as the default remote encoding
       --unlink                    remove file before clobber
       --xattr                     turn on storage of metadata in extended file attributes

Directories:
  -nd, --no-directories            don't create directories
  -x,  --force-directories         force creation of directories
  -nH, --no-host-directories       don't create host directories
       --protocol-directories      use protocol name in directories
  -P,  --directory-prefix=PREFIX   save files to PREFIX/..
       --cut-dirs=NUMBER           ignore NUMBER remote directory components

HTTP options:
       --http-user=USER            set http user to USER
       --http-password=PASS        set http password to PASS
       --no-cache                  disallow server-cached data
       --default-page=NAME         change the default page name (normally
                                     this is 'index.html'.)
  -E,  --adjust-extension          save HTML/CSS documents with proper extensions
       --ignore-length             ignore 'Content-Length' header field
       --header=STRING             insert STRING among the headers
       --compression=TYPE          choose compression, one of auto, gzip and none. (default: none)
       --max-redirect              maximum redirections allowed per page
       --proxy-user=USER           set USER as proxy username
       --proxy-password=PASS       set PASS as proxy password
       --referer=URL               include 'Referer: URL' header in HTTP request
       --save-headers              save the HTTP headers to file
  -U,  --user-agent=AGENT          identify as AGENT instead of Wget/VERSION
       --no-http-keep-alive        disable HTTP keep-alive (persistent connections)
       --no-cookies                don't use cookies
       --load-cookies=FILE         load cookies from FILE before session
       --save-cookies=FILE         save cookies to FILE after session
       --keep-session-cookies      load and save session (non-permanent) cookies
       --post-data=STRING          use the POST method; send STRING as the data
       --post-file=FILE            use the POST method; send contents of FILE
       --method=HTTPMethod         use method "HTTPMethod" in the request
       --body-data=STRING          send STRING as data. --method MUST be set
       --body-file=FILE            send contents of FILE. --method MUST be set
       --content-disposition       honor the Content-Disposition header when
                                     choosing local file names (EXPERIMENTAL)
       --content-on-error          output the received content on server errors
       --auth-no-challenge         send Basic HTTP authentication information
                                     without first waiting for the server's
                                     challenge

HTTPS (SSL/TLS) options:
       --secure-protocol=PR        choose secure protocol, one of auto, SSLv2,
                                     SSLv3, TLSv1, TLSv1_1, TLSv1_2, TLSv1_3 and PFS
       --https-only                only follow secure HTTPS links
       --no-check-certificate      don't validate the server's certificate
       --certificate=FILE          client certificate file
       --certificate-type=TYPE     client certificate type, PEM or DER
       --private-key=FILE          private key file
       --private-key-type=TYPE     private key type, PEM or DER
       --ca-certificate=FILE       file with the bundle of CAs
       --ca-directory=DIR          directory where hash list of CAs is stored
       --crl-file=FILE             file with bundle of CRLs
       --pinnedpubkey=FILE/HASHES  Public key (PEM/DER) file, or any number
                                   of base64 encoded sha256 hashes preceded by
                                   'sha256//' and separated by ';', to verify
                                   peer against
       --random-file=FILE          file with random data for seeding the SSL PRNG

       --ciphers=STR           Set the priority string (GnuTLS) or cipher list string (OpenSSL) directly.
                                   Use with care. This option overrides --secure-protocol.
                                   The format and syntax of this string depend on the specific SSL/TLS engine.
HSTS options:
       --no-hsts                   disable HSTS
       --hsts-file                 path of HSTS database (will override default)

FTP options:
       --ftp-user=USER             set ftp user to USER
       --ftp-password=PASS         set ftp password to PASS
       --no-remove-listing         don't remove '.listing' files
       --no-glob                   turn off FTP file name globbing
       --no-passive-ftp            disable the "passive" transfer mode
       --preserve-permissions      preserve remote file permissions
       --retr-symlinks             when recursing, get linked-to files (not dir)

FTPS options:
       --ftps-implicit                 use implicit FTPS (default port is 990)
       --ftps-resume-ssl               resume the SSL/TLS session started in the control connection when
                                         opening a data connection
       --ftps-clear-data-connection    cipher the control channel only; all the data will be in plaintext
       --ftps-fallback-to-ftp          fall back to FTP if FTPS is not supported in the target server
WARC options:
       --warc-file=FILENAME        save request/response data to a .warc.gz file
       --warc-header=STRING        insert STRING into the warcinfo record
       --warc-max-size=NUMBER      set maximum size of WARC files to NUMBER
       --warc-cdx                  write CDX index files
       --warc-dedup=FILENAME       do not store records listed in this CDX file
       --no-warc-compression       do not compress WARC files with GZIP
       --no-warc-digests           do not calculate SHA1 digests
       --no-warc-keep-log          do not store the log file in a WARC record
       --warc-tempdir=DIRECTORY    location for temporary files created by the
                                     WARC writer
```

```
Recursive download:
  -r,  --recursive                 specify recursive download
  -l,  --level=NUMBER              maximum recursion depth (inf or 0 for infinite)
       --delete-after              delete files locally after downloading them
  -k,  --convert-links             make links in downloaded HTML or CSS point to
                                     local files
       --convert-file-only         convert the file part of the URLs only (usually known as the basename)
       --backups=N                 before writing file X, rotate up to N backup files
  -K,  --backup-converted          before converting file X, back up as X.orig
  -m,  --mirror                    shortcut for -N -r -l inf --no-remove-listing
  -p,  --page-requisites           get all images, etc. needed to display HTML page
       --strict-comments           turn on strict (SGML) handling of HTML comments

Recursive accept/reject:
  -A,  --accept=LIST               comma-separated list of accepted extensions
  -R,  --reject=LIST               comma-separated list of rejected extensions
       --accept-regex=REGEX        regex matching accepted URLs
       --reject-regex=REGEX        regex matching rejected URLs
       --regex-type=TYPE           regex type (posix|pcre)
  -D,  --domains=LIST              comma-separated list of accepted domains
       --exclude-domains=LIST      comma-separated list of rejected domains
       --follow-ftp                follow FTP links from HTML documents
       --follow-tags=LIST          comma-separated list of followed HTML tags
       --ignore-tags=LIST          comma-separated list of ignored HTML tags
  -H,  --span-hosts                go to foreign hosts when recursive
  -L,  --relative                  follow relative links only
  -I,  --include-directories=LIST  list of allowed directories
       --trust-server-names        use the name specified by the redirection
                                     URL's last component
  -X,  --exclude-directories=LIST  list of excluded directories
  -np, --no-parent                 don't ascend to the parent directory

Email bug reports, questions, discussions to <bug-wget@gnu.org>
and/or open issues at https://savannah.gnu.org/bugs/?func=additem&group=wget.
```

```
# Let's get the sequences!
wget https://live.bio.ed.ac.uk/bioinfmsc/nem.fasta
```

```
--2025-10-01 12:40:49--  https://live.bio.ed.ac.uk/bioinfmsc/nem.fasta
Resolving live.bio.ed.ac.uk (live.bio.ed.ac.uk)... 129.215.237.166
Connecting to live.bio.ed.ac.uk (live.bio.ed.ac.uk)|129.215.237.166|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3400 (3.3K) [text/html]
Saving to: 'nem.fasta'

nem.fasta                100%[===============================================================================>]   3.32K  --.-KB/s    in 0s

2025-10-01 12:40:49 (450 MB/s) - 'nem.fasta' saved [3400/3400]
```

Just out of interest, what is the IP address of `live.bio.ed.ac.uk`, its MAC (media access control) address? It was given in the wget communication stream to our screen, but, if you ever need to find out again, this is how to do it:

```
nslookup live.bio.ed.ac.uk
```

```
Server:         129.215.205.191
Address:        129.215.205.191#53

Name:   live.bio.ed.ac.uk
Address: 129.215.237.166
```

```
nslookup bioinfmsc7.bio.ed.ac.uk
```

```
Server:         129.215.205.191
Address:        129.215.205.191#53

Name:   bioinfmsc7.bio.ed.ac.uk
Address: 129.215.237.195
```

We can also get info about the connections on our current server (example below for bioinfmsc7.bio.ed.ac.uk)?

```
ip addr
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 100
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: eno1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1
    link/ether 70:b5:e8:ef:f9:06 brd ff:ff:ff:ff:ff:ff
    altname enp225s0f0
    inet 129.215.237.195/24 metric 100 brd 129.215.237.255 scope global dynamic eno1
       valid_lft 388829sec preferred_lft 388829sec
    inet6 fe80::72b5:e8ff:feef:f906/64 scope link
       valid_lft forever preferred_lft forever
3: eno2: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc mq state DOWN group default ql
    link/ether 70:b5:e8:ef:f9:07 brd ff:ff:ff:ff:ff:ff
    altname enp225s0f1
4: ens1f0np0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc mq state DOWN group defau
    link/ether f8:f2:1e:e7:2b:70 brd ff:ff:ff:ff:ff:ff
    altname enp33s0f0np0
5: ens1f1np1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc mq state DOWN group defau
    link/ether f8:f2:1e:e7:2b:71 brd ff:ff:ff:ff:ff:ff
    altname enp33s0f1np1
6: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group de
    link/ether 02:42:b5:e3:24:91 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
```

Mini detour

Where is the msc8 server located, you wonder!? Try a web-link to get an approximate geo-location: https://www.ip-lookup.org/location/129.215.237.85
If you log in via some Broadband provider, try that link with your assigned IP address!

End of detour!

- How big is the file?

  `ls -al *.fasta`

- What does the file have in it?

  `less nem.fasta`

- How many sequences are there?

  `grep -c ">" nem.fasta`

- Are they DNA or protein? What does each sequence start with? We could write a simple script to find out!

```
cat nem.fasta | awk '{
# Is this line a fasta header (the first character is a ">")

if(substr($1,1,1)==">")
 {
print "this is a header line: " $0;
hline=FNR ; # FNR is file number record (i.e. which line of the file)
 }
# Identify the first line of the actual sequence

# ...and then the first character of that line
```

Give feedback
Ask for help

```
if(FNR==hline+1)
 {
print "First line is:" $0;
first_seq_character=substr($0,1,1)
print "First character is: " first_seq_character
print first_seq_character > "first_seq_character.txt"
 }
}'
```

```
this is a header line: >gi|407233260|gb|AFT82970.1| arginine kinase [Teladorsagia circumcincta]
First line is:MSVPPEIIKKIEDGYQTLQNAKDCHSLLKKYLTKEVVDQLKDKKTKLGATLWDVIQSGVANLDSGVGVYA
First character is: M
this is a header line: >gi|407233262|gb|AFT82971.1| arginine kinase [Haemonchus contortus]
First line is:MSVPPEIIKKIEDGYQTLQNAKDCHSLLKKYLTKEVVDQLKDKKTKLGATLWDVIQSGVANLDSGVGVYA
First character is: M
this is a header line: >gi|396578500|gb|AFN86184.1| secreted SPRY domain-containing protein 16, partial [Globodera rostochiensis]
First line is:FFLFLAAVCLLVADASPRKKTKLEKGPSSSGNAESTLGLTPQNQWAARHKDLTLTDGLIVQSIGEKREWR
First character is: F
...
```

- So, what did we find?

```
wc -l first_seq_character.txt
```

```
59832 first_seq_character.txt
```

```
sort first_seq_character.txt | uniq -c | sort -k1,1nr
```

```
  54033 M
    572 V
    547 I
    439 S
    397 G
    394 L
    373 E
    345 F
    337 N
    330 A
    328 K
    319 D
    283 T
    236 R
    224 Y
    203 Q
    153 P
    123 H
    113 C
     79 W
      4 X
```

Are the sequences DNA or protein?!

Just because it has G/A/T/C doesn't mean it is DNA.... beware! If it ONLY has GATC, then if *probably* is DNA, but we really don't know for sure from only looking at the sequence.

Make the sequence set "BLASTable" (searchable)

BLAST has a lot of command line options, and these can vary with which version of the BLAST suite of programmes is installed. For the NCBI documentation, have a look at this: https://www.ncbi.nlm.nih.gov/books/NBK279690/

The first step in the process is getting our sequences into a format that can then be searched efficiently by the BLAST programme: we use a programme called `makeblastdb`

As always, help is at hand if we type the wrong thing:

```
makeblastdb
```

```
USAGE
  makeblastdb [-h] [-help] [-help-full] [-in input_file] [-input_type type]
    -dbtype molecule_type [-title database_title] [-parse_seqids]
    [-hash_index] [-mask_data mask_data_files] [-mask_id mask_algo_ids]
    [-mask_desc mask_algo_descriptions] [-gi_mask]
    [-gi_mask_name gi_based_mask_names] [-out database_name]
    [-blastdb_version version] [-max_file_sz number_of_bytes]
    [-metadata_output_prefix ] [-logfile File_Name] [-taxid TaxID]
    [-taxid_map TaxIDMapFile] [-oid_masks oid_masks] [-version]

DESCRIPTION
   Application to create BLAST databases, version 2.17.0+

Use '-help' to print detailed descriptions of command line arguments
========================================================================

Error: Argument "dbtype".
Mandatory value is missing:  `String, `nucl', `prot''
Error:  (CArgException::eNoArg) Argument "dbtype".
Mandatory value is missing:  `String, `nucl', `prot''
```

```
# Let's make a database
makeblastdb -in nem.fasta -dbtype prot -out nem
```

```
Building a new DB, current time: 10/01/2025 14:56:15
New DB name:   /localdisk/home/aivens2/Exercises/Lecture06/nem
New DB title:  nem.fasta
Sequence type: Protein
Keep MBits: T
Maximum file size: 3000000000B
Adding sequences from FASTA; added 59832 sequences in 0.985672 seconds.
```

What has it made?

 # Lets use `-lh` ("el" "aitch") as parameters for `ls`: "lh" means long listing, human readable form

ls -lh

```
-rw-rw-r-- 1 aivens2 ansible   117K Oct  1 14:29 first_seq_character.txt
-rw-rw-r-- 1 aivens2 ansible    27M Oct  1 14:23 nem.fasta
-rw-r--r-- 1 aivens2 g_aivens2  20K Oct  1 15:11 nem.pdb
-rw-r--r-- 1 aivens2 g_aivens2 8.4M Oct  1 15:11 nem.phr
-rw-r--r-- 1 aivens2 g_aivens2 468K Oct  1 15:11 nem.pin
-rw-r--r-- 1 aivens2 g_aivens2  425 Oct  1 15:11 nem.pjs
-rw-r--r-- 1 aivens2 g_aivens2 702K Oct  1 15:11 nem.pot
-rw-r--r-- 1 aivens2 g_aivens2  21M Oct  1 15:11 nem.psq
-rw-r--r-- 1 aivens2 g_aivens2  16K Oct  1 15:11 nem.ptf
-rw-r--r-- 1 aivens2 g_aivens2 234K Oct  1 15:11 nem.pto
```

What are some of these nem files?

nem.**fasta** fasta file of 59832 sequences (database input)

nem.**phr** contains the **p**rotein sequence **h**eade**r**s (names)

nem.**pin** contains the **p**rotein **in**dex (the locations of short words)

nem.**psq** contains the compressed **p**rotein **s**e**q**uences

Give feedback
Ask for help

More flags

Tells you more about all the various options that are available

makeblastdb -help

```
USAGE
  makeblastdb [-h] [-help] [-help-full] [-in input_file] [-input_type type]
    -dbtype molecule_type [-title database_title] [-parse_seqids]
    [-hash_index] [-mask_data mask_data_files] [-mask_id mask_algo_ids]
    [-mask_desc mask_algo_descriptions] [-gi_mask]
    [-gi_mask_name gi_based_mask_names] [-out database_name]
    [-blastdb_version version] [-max_file_sz number_of_bytes]
    [-metadata_output_prefix ] [-logfile File_Name] [-taxid TaxID]
    [-taxid_map TaxIDMapFile] [-oid_masks oid_masks] [-version]

DESCRIPTION
   Application to create BLAST databases, version 2.17.0+

REQUIRED ARGUMENTS
 -dbtype <String, `nucl', `prot'>
   Molecule type of target db

OPTIONAL ARGUMENTS
 -h
   Print USAGE and DESCRIPTION;  ignore all other parameters
 -help
   Print USAGE, DESCRIPTION and ARGUMENTS; ignore all other parameters
 -help-full
   Print USAGE, DESCRIPTION and ARGUMENTS, including hidden ones; ignore all
   other parameters
 -version
   Print version number;  ignore other arguments

 *** Input options
 -in <File_In>
   Input file/database name
   Default = `-'
 -input_type <String, `asn1_bin', `asn1_txt', `blastdb', `fasta'>
   Type of the data specified in input_file
   Default = `fasta'

 *** Configuration options
 -title <String>
   Title for BLAST database
   Default = input file name provided to -in argument
 -parse_seqids
   Option to parse seqid for FASTA input if set, for all other input types
   seqids are parsed automatically
 -hash_index
   Create index of sequence hash values.

 *** Sequence masking options
 -mask_data <String>
   Comma-separated list of input files containing masking data as produced by
```

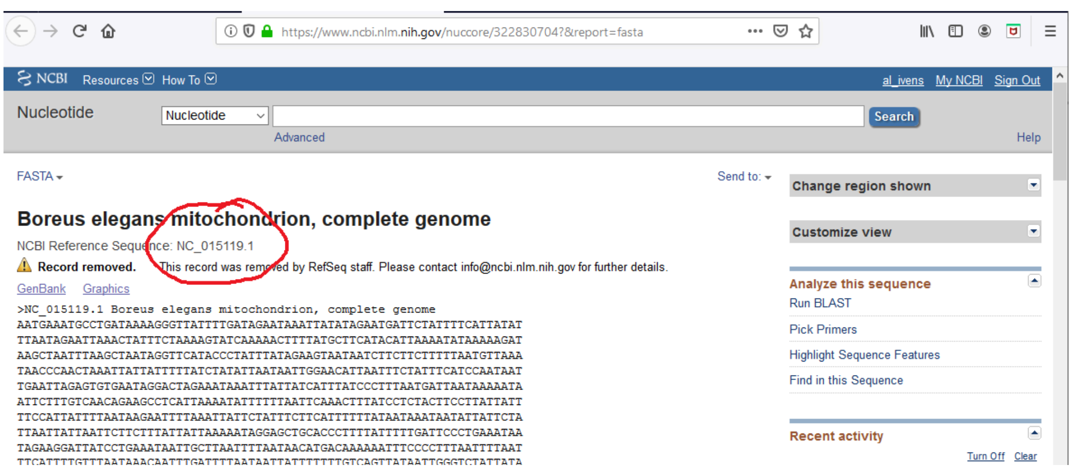
```
  NCBI masking applications (e.g. dustmasker, segmasker, windowmasker)
-mask_id <String>
  Comma-separated list of strings to uniquely identify the masking algorithm
   * Requires:  mask_data
   * Incompatible with:  gi_mask
-mask_desc <String>
  Comma-separated list of free form strings to describe the masking algorithm
  details
   * Requires:  mask_id
-gi_mask
  Create GI indexed masking data.
   * Requires:  parse_seqids
   * Incompatible with:  mask_id
-gi_mask_name <String>
  Comma-separated list of masking data output files.
   * Requires:  mask_data, gi_mask

*** Output options
-out <String>
  Name of BLAST database to be created
  Default = input file name provided to -in argumentRequired if multiple
  file(s)/database(s) are provided as input
-blastdb_version <Integer, 4..5>
  Version of BLAST database to be created
  Default = `5'
-max_file_sz <String>
  Maximum file size for BLAST database files
  Default = `3GB'
-metadata_output_prefix <String>
  Path prefix for location of database files in metadata
-logfile <File_Out>
  File to which the program log should be redirected

*** Taxonomy options
-taxid <Integer, >=0>
  Taxonomy ID to assign to all sequences
   * Incompatible with:  taxid_map
-taxid_map <File_In>
  Text file mapping sequence IDs to taxonomy IDs.
  Format:<SequenceId> <TaxonomyId><newline>
   * Requires:  parse_seqids
   * Incompatible with:  taxid
-oid_masks <Integer>
  0x01 Exclude Model
```

A real BLAST

Let's search our database using a test sequence.
We decide that we want to use a portion (nucleotide bases 1426 to 2962, for example) of a long sequence to try some things out:

https://www.ncbi.nlm.nih.gov/nuccore/322830704?&report=fasta

...but that is using the GI identifier (*322830704*, which NCBI decided to stop using a while back), gets the whole sequence which is actually 16803 bases long, AND the output is the web page html version...!



It does tell us the Accession number though (NC_015119.1)!

We could copy paste from the web page into a file we create using our favourite Linux plain text editor programme `nano` (or `vi` or ...):

but that really isn't very efficient.

We could also use the MobaXterm text editor, but that isn't ideal nor always available either...!

Give feedback
Ask for help

or we could do it all on the command line without using Firefox/Safari/Chrome/whatever with this long command:

```
wget -O testsequence.fasta "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?
db=nuccore&id=NC_015119.1&strand=1&seq_start=1426&seq_stop=2962&rettype=fasta&retmode=te
```

Use `wget` specifying:

- -O testsequence.fasta (the **O**utput file)
- efetch.fcgi? (there are many NCBI utilities, described in more detail at https://www.ncbi.nlm.nih.gov/books/NBK25501/)
- db=nuccore (which database)
- id=NC_015119.1 (sequence AccessionID)
- strand=1 (top strand)
- seq_start=1426 (start from base 1426 of the sequence)
- seq_stop=2962 (stop at base 2962 of the sequence)
- rettype=fasta (return in fasta format)
- retmode=text (return in plain text format please!)

The `wget` method that we used to get our database sequence set is far easier to scale up as part of a script/programme/workflow.

Run the command version 1

wget -O testsequence.fasta "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nuccore&id=NC_015119.1&strand=1&seq_start=1426&seq_stop=2962&rettype=fasta&retmode=te

```
--2025-10-01 15:00:07--  https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?
Resolving eutils.ncbi.nlm.nih.gov (eutils.ncbi.nlm.nih.gov)... 130.14.29.110, 2607:
Connecting to eutils.ncbi.nlm.nih.gov (eutils.ncbi.nlm.nih.gov)|130.14.29.110|:443.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [text/plain]
Saving to: 'testsequence.fasta'

testsequence.fasta                        [ <=>

2025-10-01 15:00:08 (15.6 MB/s) - 'testsequence.fasta' saved [1629]
```

Run the command version 2

weblink="https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nuccore&id=NC_015119.1&strand=1&seq_start=1426&seq_stop=2962&rettype=fasta&retmode=te

echo ${weblink}

https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nuccore&id=NC_015119.1&strand=1&seq_start=1426&seq_stop=2962&rettype=fasta&retmode=te

wget -O link_testsequence.fasta ${weblink}

```
--2025-10-01 15:01:33--  https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nucc
Resolving eutils.ncbi.nlm.nih.gov (eutils.ncbi.nlm.nih.gov)... 130.14.29.110, 2607:f220:41
Connecting to eutils.ncbi.nlm.nih.gov (eutils.ncbi.nlm.nih.gov)|130.14.29.110|:443... conn
HTTP request sent, awaiting response... 200 OK
Length: unspecified [text/plain]
Saving to: 'link_testsequence.fasta'

link_testsequence.fasta                 [ <=>

2025-10-01 15:01:33 (15.4 MB/s) - 'link_testsequence.fasta' saved [1629]
```

Run the command version 3

saved_filename="saved_testsequence.fasta"

wget -O ${saved_filename} ${weblink}

```
--2025-10-01 15:02:18--  https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nucc
Resolving eutils.ncbi.nlm.nih.gov (eutils.ncbi.nlm.nih.gov)... 130.14.29.110, 2607:f220:41
Connecting to eutils.ncbi.nlm.nih.gov (eutils.ncbi.nlm.nih.gov)|130.14.29.110|:443... conn
HTTP request sent, awaiting response... 200 OK
Length: unspecified [text/plain]
Saving to: 'saved_testsequence.fasta'

saved_testsequence.fasta                [ <=>
```

```
2025-10-01 15:02:18 (14.9 MB/s) - ‘saved_testsequence.fasta’ saved [1629]
```

What testsequence files do we have?

ls -1 *sequence*

```
link_testsequence.fasta
saved_testsequence.fasta
testsequence.fasta
```

How big are they?

du -h *sequence*

```
4.0K    link_testsequence.fasta
4.0K    saved_testsequence.fasta
4.0K    testsequence.fasta
```

What files do we have there now?

ls -al

```
total 58228
drwxrwxr-x 2 aivens2 ansible       319 Oct  1 15:15 .
drwxr-xr-x 6 aivens2 g_aivens2     146 Oct  1 12:39 ..
-rw-rw-r-- 1 aivens2 ansible    119664 Oct  1 14:29 first_seq_character.txt
-rw-r--r-- 1 aivens2 g_aivens2    1629 Oct  1 15:15 link_testsequence.fasta
-rw-rw-r-- 1 aivens2 ansible  27303735 Oct  1 14:23 nem.fasta
-rw-r--r-- 1 aivens2 g_aivens2   20480 Oct  1 15:11 nem.pdb
-rw-r--r-- 1 aivens2 g_aivens2 8775689 Oct  1 15:11 nem.phr
-rw-r--r-- 1 aivens2 g_aivens2  478744 Oct  1 15:11 nem.pin
-rw-r--r-- 1 aivens2 g_aivens2     425 Oct  1 15:11 nem.pjs
-rw-r--r-- 1 aivens2 g_aivens2  717992 Oct  1 15:11 nem.pot
-rw-r--r-- 1 aivens2 g_aivens2 21923788 Oct  1 15:11 nem.psq
-rw-r--r-- 1 aivens2 g_aivens2   16384 Oct  1 15:11 nem.ptf
-rw-r--r-- 1 aivens2 g_aivens2  239332 Oct  1 15:11 nem.pto
-rw-r--r-- 1 aivens2 g_aivens2    1629 Oct  1 15:15 saved_testsequence.fasta
-rw-r--r-- 1 aivens2 g_aivens2    1629 Oct  1 15:15 testsequence.fasta
```

The BLAST command itself

We have a DNA query and a protein database, so **blastx** is the flavour of BLAST needed

 # Runs `blastx` of our testsequence.fasta against the nem database, using default parameters, to the screen
blastx -db nem -query testsequence.fasta

```
BLASTX 2.17.0+


Reference: Stephen F. Altschul, Thomas L. Madden, Alejandro A.
Schaffer, Jinghui Zhang, Zheng Zhang, Webb Miller, and David J.
Lipman (1997), "Gapped BLAST and PSI-BLAST: a new generation of
protein database search programs", Nucleic Acids Res. 25:3389-3402.



Database: nem.fasta
           59,832 sequences; 21,863,955 total letters



Query= NC_015119.1:1426-2962 Boreus elegans mitochondrion, complete genome

Length=1537
                                                                      Score     E
Sequences producing significant alignments:                          (Bits)  Value

gi|225622197|ref|YP_002725710.1| cytochrome c oxidase subunit I [...  432     7e-147
gi|288903410|ref|YP_003434131.1| cytochrome c oxidase subunit I [...  430     3e-146
...
```

or better still:

 # Runs `blastx` of our testsequence.fasta against the nem database, using default parameters, saved to the file `blastoutput1.out`
blastx -db nem -query testsequence.fasta > blastoutput1.out

... a couple of seconds...

have a look at the output

more blastoutput1.out

```
BLASTX 2.17.0+


Reference: Stephen F. Altschul, Thomas L. Madden, Alejandro A.
Schaffer, Jinghui Zhang, Zheng Zhang, Webb Miller, and David J.
Lipman (1997), "Gapped BLAST and PSI-BLAST: a new generation of
protein database search programs", Nucleic Acids Res. 25:3389-3402.



Database: nem.fasta
           59,832 sequences; 21,863,955 total letters



Query= NC_015119.1:1426-2962 Boreus elegans mitochondrion, complete genome

Length=1537
                                                                      Score     E
Sequences producing significant alignments:                          (Bits)  Value

gi|225622197|ref|YP_002725710.1| cytochrome c oxidase subunit I [...  432     7e-14
gi|288903410|ref|YP_003434131.1| cytochrome c oxidase subunit I [...  430     3e-14
....
....
gi|402583242|gb|EJW77186.1| cytochrome c oxidase subunit 1, parti...  34.7    0.19
gi|324575316|gb|ADY49947.1| Cytochrome c oxidase subunit 1 protei...  29.3    1.4
gi|402583579|gb|EJW77523.1| hypothetical protein WUBG_11566 [Wuch...  30.4    5.6


>gi|225622197|ref|YP_002725710.1| cytochrome c oxidase subunit
I [Ancylostoma caninum]
Length=525

 Score = 432 bits (1110),  Expect = 7e-147, Method: Compositional matrix adjust.
 Identities = 274/444 (62%), Positives = 352/444 (79%), Gaps = 1/444 (0%)
 Frame = +1

Query  4     QKWLLSTNHKDIGTLYFIFGT*SGIIGTSLRILIRTELRHPGSIIRDDQIFNvivtahaf  183
               WL S+NHKDIGTLYF+FG  SG++GTSL ++IR EL  PG ++ + Q++N I+TAHA
Sbjct  12    ATWLESSNHKDIGTLYFLFGLWSGMVGTSLSLIIRLELAKPGLLLGNGQLYNSIITAHAI  71

Query  184   viiffivipiliggfgN*LVPLILGAPDIAFPRINNIRF*llppsltlllIGSMVENGAG  363
             ++IFF+V+P +IGGFGN +VPL+LGAPD++FPR+NN+ F LLP ++ L+L    V+ G G
Sbjct  72    LMIFFMVMPTMIGGFGNWMVPLMLGAPDMSFPRLNNLSFWLLPTAMFLILDSCFVDMGCG  131

Query  364   TG*TVYPPLSSYIAHAGSSVDLTIFSLHLAGISSILGAINFITTVINIRPQRITLDRIPL  543
             T  TVYPPLS+ + H GSSVDL IFSLH AG+SSILG INF+ T  N+R   I+L+ + L
Sbjct  132   TSWTVYPPLST-LGHPGSSVDLAIFSLHCAGLSSILGGINFMCTTKNLRSSSISLEHMSL  190
....
....
Lambda      K        H        a         alpha
   0.318    0.134    0.401    0.792     4.96

Gapped
Lambda      K        H        a         alpha    sigma
   0.267   0.0410    0.140     1.90     42.6     43.6

Effective search space used: 6262082055
```

Give feedback
Ask for help

```
  Database: nem.fasta
    Posted date:  Oct 1, 2025  3:16 PM
  Number of letters in database: 21,863,955
  Number of sequences in database:  59,832



Matrix: BLOSUM62
Gap Penalties: Existence: 11, Extension: 1
Neighboring words threshold: 12
Window for multiple hits: 40
```

```
# Nice, but we want structure for parsing the data...!
# We can change the output format easily
blastx -db nem \
-query testsequence.fasta \
-outfmt 7 > blastoutput2.out
```

The above command runs `blastx` of our testsequence.fasta against the nem BLAST database, using default parameters, saved in a tabular format, to the file `blastoutput2.out`

BLAST output

What does our new output format look like? Look familiar?!

head blastoutput2.out

```
# BLASTX 2.17.0+
# Query: NC_015119.1:1426-2962 Boreus elegans mitochondrion, complete genome
# Database: nem
# Fields: query acc.ver, subject acc.ver, % identity, alignment length, mismatches,
# 405 hits found
NC_015119.1:1426-2962   gi|225622197|ref|YP_002725710.1|        61.712  444     169
NC_015119.1:1426-2962   gi|288903410|ref|YP_003434131.1|        61.991  442     167
NC_015119.1:1426-2962   gi|326536058|ref|YP_004300493.1|        62.443  442     165
NC_015119.1:1426-2962   gi|188011119|ref|YP_001905892.1|        61.312  442     170
NC_015119.1:1426-2962   gi|225622184|ref|YP_002725698.1|        61.486  444     170
```

Key features to notice

- comment lines start with #
- fields in the comment section are separated by a space
- we are told what is in each of the "data section" fields
- each "data section" line is a HSP
- fields in the "data section" are separated by tabs

Much smaller on the hard disk too

wc -l blastoutput*

```
  7979 blastoutput1.out
   411 blastoutput2.out
  8390 total
```

Surely there must be a better way to do this? Yes, there is, and we will learn how to do it very soon...

Meanwhile, it is time to sort, filter and manipulate these data using our bash scripting skills!

Exercises

How to log in to bioinfmsc7 or bioinfmsc8 (**not** bioinfmsc9)...
...and use git...
...to save your bash and awk scripts...
...written with Unix/awk commands...
...other Bash help...
..and arrays might be useful...

An alternative/additional "all about BLAST" (quite old, but easy reading)

Your task(s):

1. create a directory in Exercises for today's exercises, and change directory to it
2. download the `nem.fasta` file and create the blastable database as shown in the lecture
3. get the same query sequence as we did earlier in today's lecture and save it as `testsequence.fasta`
4. generate the `blastoutput2.out` output
5. how can you do a remote blast of your query against a database at NCBI (without downloading the database from NCBI, that is!!)?
6. add and commit the above files to your git repository with a suitable message
7. Write a `bash` script that will:
    - list the subject accession for all HSPs
    - list the alignment length and percent ID for all HSPs
    - show the HSPs with more than 20 mismatches
    - show the HSPs shorter than 100 amino acids and with more than 20 mismatches

- list the first 20 HSPs that have fewer than 20 mismatches
- how many HSPs are shorter than 100 amino acids?
- list the top ten highest (best) HSPs.
- list the start positions of all matches where the HSP Subject accession includes the letters string "AEI"
- how many subject sequences have more than one HSP?
- what percentage of each HSP is made up of mismatches?
- allocate HSPs into different groups based on their scores

8. You could try answering them using awk too, once you have done it in bash if you would like more practice at exercises...

Don't forget to add and commit the relevant files to your git repository with an appropriate message

Give feedback
Ask for help

Sending email from the server

BEFORE we start the exercises, today we are going to send an email from the bioinformatics bioinfmsc7 or bioinfmsc8 server (whichever you are on!)
The programme we are going to use is called `mail`.
You are going to use it to send me a short message with some information from your Lecture Exercises git log.

Here is what you need to do, and we are going to do it today, now, before you start anything else!

1. log into the server bioinfmsc7 or bioinfmsc8 (**not** bioinfmsc9)...
2. change directory to wherever you have your **Lecture Exercises git repository** (NOT the optional ICA MyFirstPipeline one)!
3. copy-paste the following 4 lines of commands into your terminal window

```
cd ${HOME}/Exercises
DATE=$(date)
git log | grep "Date:" > message_to_al
mail -s "$USER $HOSTNAME $DATE" < message_to_al aivens2@ed.ac.uk
cat message_to_al
```

Mail sent, all done, thanks!

Enjoy!

Exercise answers: possible solutions using bash

Don't forget to add and commit the relevant files to your git repository with an appropriate message

These might include the blastoutput2.out blast output file, and your script(s)?

```
# Initial stuff
```

```
# Get the sequences to make the database
wget https://live.bio.ed.ac.uk/bioinfmsc/nem.fasta
```

```
# Make the blastable database
makeblastdb -in nem.fasta -dbtype prot -out nem
```

```
# Get the query sequence (could use the other methods, with variables)
wget -O testsequence.fasta "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nuccore&id=NC_015119.1&strand=1&seq_start=1426&seq_stop=2962&rettype=fasta&retmode=te
```

```
# Run the BLAST #1
blastx -db nem -query testsequence.fasta > blastoutput1.out
```

```
# Run the BLAST #2
blastx -db nem -query testsequence.fasta -outfmt 7 > blastoutput2.out
```

How can you do a remote blast of your query against a database at NCBI (without downloading the database from NCBI, that is!!)?

```
# Get ready for git!
git lo
```

```
19cdd2d (HEAD -> master) Added the example_people_data.tsv file from Lecture03
137c3db Decided to keep blastoutput2.out
08d379b Decided to keep myinputfile.tsv
c9b5b18 First attempt at parsing BLAST data
bbf825c My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
```

```
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeea I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```

git st

```
On branch master
Changes not staged for commit:
  (use "git add/rm ..." to update what will be committed)
  (use "git restore ..." to discard changes in working directory)
        deleted:    ../Lecture04/CodeFiles.tar.gz

Untracked files:
  (use "git add ..." to include in what will be committed)
        ../BN.out
        ../BN.sh
        ../Lecture04/Als_ICA/
        ../Lecture04/all_the_things_I_did
        ../Lecture04/outs/
        ../Lecture05/Country.Mozambique.details
        ./

no changes added to commit (use "git add" and/or "git commit -a")
```

Add and commit all the files to our repo

git add *

Check the status BEFORE COMMIT STEP

git st

```
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   blastoutput1.out
        new file:   blastoutput2.out
        new file:   first_seq_character.txt
        new file:   link_testsequence.fasta
        new file:   nem.fasta
        new file:   nem.pdb
        new file:   nem.phr
        new file:   nem.pin
        new file:   nem.pjs
        new file:   nem.pot
        new file:   nem.psq
        new file:   nem.ptf
        new file:   nem.pto
        new file:   saved_testsequence.fasta
        new file:   testsequence.fasta

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    ../Lecture04/CodeFiles.tar.gz
```

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ../BN.out
        ../BN.sh
        ../Lecture04/Als_ICA/
        ../Lecture04/all_the_things_I_did
        ../Lecture04/outs/
        ../Lecture05/Country.Mozambique.details
```

 # Commit with a message

git commit -m "nematode BLASTDB stuff added"

```
[master ce2c15a] nematode BLASTDB stuff added
 15 files changed, 529331 insertions(+)
 create mode 100644 Lecture06/blastoutput1.out
 create mode 100644 Lecture06/blastoutput2.out
 create mode 100644 Lecture06/first_seq_character.txt
 create mode 100644 Lecture06/link_testsequence.fasta
 create mode 100644 Lecture06/nem.fasta
 create mode 100644 Lecture06/nem.pdb
 create mode 100644 Lecture06/nem.phr
 create mode 100644 Lecture06/nem.pin
 create mode 100644 Lecture06/nem.pjs
 create mode 100644 Lecture06/nem.pot
 create mode 100644 Lecture06/nem.psq
 create mode 100644 Lecture06/nem.ptf
 create mode 100644 Lecture06/nem.pto
 create mode 100644 Lecture06/saved_testsequence.fasta
 create mode 100644 Lecture06/testsequence.fasta
```

 # GitHub: create a repository in your personal account on GitHub
 # Doing it this way, you will need <u>your</u> personal account GitHub name and token ALREADY stored as variables!

echo ${mypersonalGitHub_username}

```
someusername
```

echo ${mypersonalgittoken}

```
ghp_mKY....
```

curl -u ${mypersonalGitHub_username}:${mypersonalgittoken} https://api.github.com/user/repos -d '{"name":"BPSM_Lecture06"}'

```
{
  "id": 544893198,
  "node_id": "R_kgDOIHppDg",
  "name": "BPSM_Lecture06",
  "full_name": "B123456-2121/BPSM_Lecture06",
  "private": false,
...
```

 # Tell git about our repository on GitHub, and that
 # we are calling it "Lecture06" (or something!)

git remote add Lecture06 https://github.com/${mypersonalGitHub_username}/BPSM_Lecture06.git

 # What remote links do we have now?

git remote

```
Lecture06
origin
```

 # What was the `origin` link to, I really can't remember...!?

git remote get-url origin

https://github.com/B123456-2121/MyFirstPipeline.git

 # Push our repository (called master) off to our repository on GitHub
 # using your own user and token

git push -u Lecture06 master

Username for 'https://github.com':

B123456-2121

Password for 'https://B123456-2121@github.com'

ghp_mKY....

```
Enumerating objects: 93, done.
Counting objects: 100% (93/93), done.
Delta compression using up to 256 threads
Compressing objects: 100% (86/86), done.
Writing objects: 100% (93/93), 26.46 MiB | 1.23 MiB/s, done.
Total 93 (delta 26), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (26/26), done.
To https://github.com/B123456-2121/BPSM_Lecture06.git
 * [new branch]      master -> master
branch 'master' set up to track 'Lecture06/master'.
```

 # Do the usual checks....

git st

```
On branch master
Your branch is up-to-date with 'Lecture06/master'.

Changes not staged for commit:
  (use "git add/rm ..." to update what will be committed)
  (use "git restore ..." to discard changes in working directory)
        deleted:    ../Lecture04/CodeFiles.tar.gz

Untracked files:
  (use "git add ..." to include in what will be committed)
        ../BN.out
        ../BN.sh
        ../Lecture04/Als_ICA/
        ../Lecture04/all_the_things_I_did
        ../Lecture04/outs/
        ../Lecture05/Country.Mozambique.details

no changes added to commit (use "git add" and/or "git commit -a")
```

git lo

```
ce2c15a (HEAD -> master, Lecture06/master) nematode BLASTDB stuff added
19cdd2d Added the example_people_data.tsv file from Lecture03
137c3db Decided to keep blastoutput2.out
08d379b Decided to keep myinputfile.tsv
```

Give feedback
Ask for help

```
c9b5b18 First attempt at parsing BLAST data
bbf825c My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeea I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```

All good we hope, so now let's carry on....

We need to use the tabulated form of the BLAST output, which we saved as `blastoutput2.out`, as it is well structured with minimal "unwanted" information.

 # Check the output

head -n7 blastoutput2.out

```
# BLASTX 2.17.0+
# Query: NC_015119.1:1426-2962 Boreus elegans mitochondrion, complete genome
# Database: nem
# Fields: query acc.ver, subject acc.ver, % identity, alignment length, mismatches,
# 405 hits found
NC_015119.1:1426-2962   gi|225622197|ref|YP_002725710.1|        61.712  444     169
NC_015119.1:1426-2962   gi|288903410|ref|YP_003434131.1|        61.991  442     167
```

From the "Fields" line, we can see the names of the fields, but we wont actually need, for this analysis, the hashed comment lines. We *could* just get rid of them using the following command:

grep -v "#" blastoutput2.out > HSP_lines_only.out

However, we will continue with the original file for this exercise!

 # This time, we will specify our input file using a variable to hold the filename

inputfile="$PWD/blastoutput2.out"

echo -e ${inputfile}

/home/s0000000/Exercises/Lecture06/blastoutput2.out

Have we done the input--process--output thing (a.k.a figured out what we need to do to actually answer the questions!?)

 # Yes? OK, now start writing a script in our favourite plain text editor
 # I decided to call it `run_this_script_v1`

 # Where is bash kept?

which bash

/usr/bin/bash

START OF SCRIPT

#!/usr/bin/bash

 # Delete any exercise output files from previous runs

rm -f *.exercise.out

 # Initial variable values at start

```
goodlines=$(grep -v "#" ${inputfile} | wc -l | cut -d ' ' -f1)
unset IFS
unset dataline
shortHSP=0;
hspcounter=0;
echo -e "We have ${goodlines} data lines for processing...\n"
```

 # Initialise a bash array for multiple HSPs, called it dupS_acc

dupS_acc=()

 # HSP score cuts chosen, and their output files

```
group1cut=150
group2cut=250
group3cut=350
outfile1="HSPscore.${group1cut}.exercise.out"
outfile2="HSPscore.${group2cut}.exercise.out"
outfile3="HSPscore.${group3cut}.exercise.out"
outfile4="HSPscore.morethan.${group3cut}.exercise.out"
```

 # Remove any existing versions of the output <u>files</u>
 # This does <u>NOT</u> remove the variables!

rm -f ${outfile1} ${outfile2} ${outfile3} ${outfile4}

 # Read in the whole line at first

```
while read wholeline
do
#  echo "Line is ${wholeline}"
```

Don't want the input lines that start with

Uses a substring notation: offset from first character, length wanted

```
if test ${wholeline:0:1} != "#"
 then
 dataline=$((dataline+1))
 # echo "Line ${dataline} starts with ${wholeline:0:1}"
```

Split the line into the fields we want

```
read Q_acc S_acc pc_identity alignment_length mismatches gap_opens Q_start Q_end S_start S_end evalue bitscore <<< ${wholeline}
```

We need to ensure that `${bitscore}` is an integer number not a real/float!

Remember, bash CAN'T deal properly with non-integers, so make it an integer

how to format outputs using `printf`

```
bitscore=$(printf "%.0f\n" ${bitscore})
```

list the subject accession for all HSPs

```
echo -e "${dataline}\t${Q_acc}\t${S_acc}" >> Subject_accessions.exercise.out
```

List the alignment length and percent ID for all HSPs

```
echo -e "${dataline}\t${alignment_length}\t${pc_identity}" >> al_leng_pcID.exercise.out
```

Show the HSPs with more than 20 mismatches

```
if test ${mismatches} -gt 20
 then
 echo -e "${dataline}\tmore than 20 mismatches:\t${Q_acc} ${S_acc} ${mismatches}"
fi
```

show the HSPs shorter than 100 amino acids and with more than 20 mismatches

Are the alignment length amino acids or nucleotides!? aa

```
if test ${alignment_length} -lt 100 && test ${mismatches} -gt 20
 then
 echo -e "${dataline}\tHSP shorter than 100aa, more \
       than 20 mismatches:\t${alignment_length}\t${mismatches}"
fi
```

list the first 20 HSPs that have fewer than 20 mismatches

```
if test ${mismatches} -lt 20
 then
 hspcounter=$((hspcounter+1))
 if test ${hspcounter} -le 20
```

Give feedback
Ask for help

```
  then
  hsp_array+=${wholeline}
  echo -e "${dataline}\t${hspcounter}\t${wholeline}" >> Fewer.than20MM.exercise.out
 fi
fi
```

how many HSPs are shorter than 100 amino acids?

```
if test ${alignment_length} -lt 100
 then
 shortHSP=$((shortHSP+1))
fi
```

list the top ten highest (best) HSPs.

BLAST output default is to give the best first

```
if test ${dataline} -le 10
 then
 echo -e "${dataline}\t${wholeline}" >> Top10.HSPs.exercise.out
fi
```

list the start positions of all matches where the
HSP Subject accession includes the letters string AEI.

```
if [[ ${S_acc} == *"AEI"* ]]; then
 echo -e "${dataline}\t${S_acc} contains AEI: \
    Subject starts at ${S_start}, Query starts at ${Q_start}" \
    >> AEIinSubjectAcc.starts.exercise.out
fi
```

how many subject sequences have more than one HSP?

```
if test ${S_acc} == ${pre_acc}
 then
 dupecount=$((dupecount+1))
 if [[ dupecount == 1 ]]; then
  dupS_acc=${S_acc}
 fi
```

Some might have more than 2: use wildcard pattern matching to see
if we have it already

```
 if [[ $dupS_acc == *${S_acc}* ]]; then
  echo ""
  else
  dupS_acc+=(${S_acc})
 fi

fi
pre_acc=${S_acc}
```

what percentage of each HSP is made up of mismatches? This is a rounded
number as bash doesn't do floating point maths... =-(

```
MMpercent=$((100*${mismatches}/${alignment_length}))
echo -e "${dataline}\t${alignment_length}\t${mismatches}\t${MMpercent}" \
   >> Mismatchpercent.exercise.out
```

allocate HSPs into different groups based on their scores

```
scorebin=1
if [ ${bitscore} -gt ${group3cut} ]; then
     scorebin=4
fi
if [ ${bitscore} -le ${group3cut} ] && [ ${bitscore} -gt ${group2cut} ]; then
     scorebin=3
fi
if [ ${bitscore} -le ${group2cut} ] && [ ${bitscore} -gt ${group1cut} ]; then
     scorebin=2
fi
```

Use a `case` statement to send the output to the relevant file
See this link for more examples

```
scoregroupdetails=$(echo -e "${dataline}\t${Q_acc}\t${S_acc}\t${bitscore}")
case $scorebin in
  4)
    echo -e "${scoregroupdetails}" >> ${outfile4}
    ;;
  3)
    echo -e "${scoregroupdetails}" >> ${outfile3}
    ;;
  2)
    echo -e "${scoregroupdetails}" >> ${outfile2}
    ;;
  1)
    echo -e "${scoregroupdetails}" >> ${outfile1}
    ;;
esac
```

END BLOCK EQUIVALENT

```
if test ${dataline} -eq ${goodlines}
 then
 echo -e "\n\nENDBLOCK\n\nThere were ${shortHSP} HSPs shorter than 100 amino acids"
 echo -e "There were ${#dupS_acc[@]} Subjects with multiple HSPs"
fi
fi   # was not a commented line in the blast data
```

The input file gets brought in here, it's a variable whose value
we can change if we want to use a different file as input...

```
done < ${inputfile}
```

END OF SCRIPT

I saved the script as `run_this_script_v1`

Time to add it to our git repo

```
git add run_this_script_v1
```

```
git commit -m "Added run_this_script_v1 from BPSM Lecture06"
```

Give feedback
Ask for help

```
[master fbe5cc0] Added run_this_script_v1 from BPSM Lecture06
 1 file changed, 168 insertions(+)
 create mode 100644 Lecture06/run_this_script_v1
```

Usual checks....

git st

```
On branch master
Your branch is ahead of 'Lecture06/master' by 1 commit.
  (use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add/rm ..." to update what will be committed)
  (use "git restore ..." to discard changes in working directory)
        deleted:    ../Lecture04/CodeFiles.tar.gz

Untracked files:
  (use "git add ..." to include in what will be committed)
        ../BN.out
        ../BN.sh
        ../Lecture04/Als_ICA/
        ../Lecture04/all_the_things_I_did
        ../Lecture04/outs/
        ../Lecture05/Country.Mozambique.details

no changes added to commit (use "git add" and/or "git commit -a")
```

git lo

```
fbe5cc0 (HEAD -> master) Added run_this_script_v1 from BPSM Lecture06
ce2c15a (Lecture06/master) nematode BLASTDB stuff added
19cdd2d Added the example_people_data.tsv file from Lecture03
137c3db Decided to keep blastoutput2.out
08d379b Decided to keep myinputfile.tsv
c9b5b18 First attempt at parsing BLAST data
bbf825c My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeea I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```

Time to push our local repository to GitHub

git push -u Lecture06 master

```
Username for 'https://github.com': B123456-2121
Password for 'https://B123456-2121@github.com':
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 256 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 2.25 KiB | 2.25 MiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/B123456-2121/BPSM_Lecture06.git
```

```
   ce2c15a..fbe5cc0  master -> master
branch 'master' set up to track 'Lecture06/master'.
```

 # Now its saved, let's run it!

chmod 755 run_this_script_v1

./run_this_script_v1

Nothing seems to happen, even if I press return!? Huh?

kill it using CTRL-C keyboard combination....

See https://ss64.com/bash/source.html where is says:

> "*When a script is run using source it runs within the existing shell, any variables created or modified by the script will remain available after the script completes. In contrast if the script is run just as `filename`, then a separate subshell (with a completely separate set of variables) would be spawned to run the script.*"

.... so, it doesn't know what `${inputfile}` is, so it doesn't work...!

 # Use `source run_this_script_v1`
 # or `. run_this_script_v1`

source run_this_script_v1

```
We have 405 data lines for processing...

1       more than 20 mismatches:        NC_015119.1:1426-2962 gi|225622197|ref|YP_00272571
2       more than 20 mismatches:        NC_015119.1:1426-2962 gi|288903410|ref|YP_00343413
3       more than 20 mismatches:        NC_015119.1:1426-2962 gi|326536058|ref|YP_00430049
4       more than 20 mismatches:        NC_015119.1:1426-2962 gi|188011119|ref|YP_00190589
5       more than 20 mismatches:        NC_015119.1:1426-2962 gi|225622184|ref|YP_00272569
6       more than 20 mismatches:        NC_015119.1:1426-2962 gi|171260186|ref|YP_00179539
7       more than 20 mismatches:        NC_015119.1:1426-2962 gi|288903312|ref|YP_00343404
8       more than 20 mismatches:        NC_015119.1:1426-2962 gi|49146527|ref|YP_026087.1|

....
399     HSP shorter than 100aa, more than 20 mismatches:        81      28
400     more than 20 mismatches:        NC_015119.1:1426-2962 gi|402169024|emb|CCK73368.1|
400     HSP shorter than 100aa, more than 20 mismatches:        51      27
401     more than 20 mismatches:        NC_015119.1:1426-2962 gi|324575366|gb|ADY49962.1|
401     HSP shorter than 100aa, more than 20 mismatches:        44      22
402     more than 20 mismatches:        NC_015119.1:1426-2962 gi|402575841|gb|EJW69801.1|
402     HSP shorter than 100aa, more than 20 mismatches:        73      43
403     more than 20 mismatches:        NC_015119.1:1426-2962 gi|402583242|gb|EJW77186.1|
403     HSP shorter than 100aa, more than 20 mismatches:        86      24


ENDBLOCK

There were 31 HSPs shorter than 100 amino acids
There were 23 Subjects with multiple HSPs
```

Give feedback
Ask for help

What was in all the saved files?

head -3 *.exercise.out

```
==> AEIinSubjectAcc.starts.exercise.out <==
353     gi|336287921|gb|AEI30249.1| contains AEI: Subject starts at 2, Query starts at 238
354     gi|336287921|gb|AEI30249.1| contains AEI: Subject starts at 136, Query starts at 6
355     gi|336287923|gb|AEI30250.1| contains AEI: Subject starts at 2, Query starts at 238

==> al_leng_pcID.exercise.out <==
1       444     61.712
2       442     61.991
3       442     62.443

==> Fewer.than20MM.exercise.out <==
354     1       NC_015119.1:1426-2962   gi|336287921|gb|AEI30249.1|     47.222  36      17
356     2       NC_015119.1:1426-2962   gi|336287923|gb|AEI30250.1|     47.222  36      17
358     3       NC_015119.1:1426-2962   gi|336287885|gb|AEI30231.1|     47.222  36      17

==> HSPscore.150.exercise.out <==
210     NC_015119.1:1426-2962   gi|392282155|gb|AFM53696.1|     142
211     NC_015119.1:1426-2962   gi|338970774|gb|AEJ33700.1|     139
212     NC_015119.1:1426-2962   gi|338970778|gb|AEJ33702.1|     139

==> HSPscore.250.exercise.out <==
86      NC_015119.1:1426-2962   gi|224176451|dbj|BAH23580.1|    236
87      NC_015119.1:1426-2962   gi|224176459|dbj|BAH23582.1|    236
88      NC_015119.1:1426-2962   gi|224176447|dbj|BAH23579.1|    236

==> HSPscore.350.exercise.out <==
59      NC_015119.1:1426-2962   gi|119637801|ref|YP_918966.1|   350
60      NC_015119.1:1426-2962   gi|119360188|ref|YP_913163.1|   344
61      NC_015119.1:1426-2962   gi|119655310|ref|YP_918977.1|   344

==> HSPscore.morethan350.exercise.out <==
1       NC_015119.1:1426-2962   gi|225622197|ref|YP_002725710.1|        432
2       NC_015119.1:1426-2962   gi|288903410|ref|YP_003434131.1|        430
3       NC_015119.1:1426-2962   gi|326536058|ref|YP_004300493.1|        430

==> Mismatchpercent.exercise.out <==
1       444     169     38
2       442     167     37
3       442     165     37

==> Subject_accessions.exercise.out <==
1       NC_015119.1:1426-2962   gi|225622197|ref|YP_002725710.1|
2       NC_015119.1:1426-2962   gi|288903410|ref|YP_003434131.1|
3       NC_015119.1:1426-2962   gi|326536058|ref|YP_004300493.1|

==> Top10.HSPs.exercise.out <==
1       NC_015119.1:1426-2962   gi|225622197|ref|YP_002725710.1|        61.712  444     16
2       NC_015119.1:1426-2962   gi|288903410|ref|YP_003434131.1|        61.991  442     16
3       NC_015119.1:1426-2962   gi|326536058|ref|YP_004300493.1|        62.443  442     16
```

git time!

git st

```
On branch master
Your branch is up-to-date with 'Lecture06/master'.

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    ../Lecture04/CodeFiles.tar.gz
```

```
        modified:   run_this_script_v1

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ../BN.out
        ../BN.sh
        ../Lecture04/Als_ICA/
        ../Lecture04/all_the_things_I_did
        ../Lecture04/outs/
        ../Lecture05/Country.Mozambique.details
        AEIinSubjectAcc.starts.exercise.out
        Fewer.than20MM.exercise.out
        HSPscore.150.exercise.out
        HSPscore.250.exercise.out
        HSPscore.350.exercise.out
        HSPscore.morethan.350.exercise.out
        Mismatchpercent.exercise.out
        Subject_accessions.exercise.out
        Top10.HSPs.exercise.out
        al_leng_pcID.exercise.out
```

 # Decided to keep all the output files (why?)

git add *.out

git commit -m "Kept bash script output files from Lecture 06"

```
[master eea4f5e] Kept bash script output files from Lecture 06
 10 files changed, 1694 insertions(+)
 create mode 100644 Lecture06/AEIinSubjectAcc.starts.exercise.out
 create mode 100644 Lecture06/Fewer.than20MM.exercise.out
 create mode 100644 Lecture06/HSPscore.150.exercise.out
 create mode 100644 Lecture06/HSPscore.250.exercise.out
 create mode 100644 Lecture06/HSPscore.350.exercise.out
 create mode 100644 Lecture06/HSPscore.morethan.350.exercise.out
 create mode 100644 Lecture06/Mismatchpercent.exercise.out
 create mode 100644 Lecture06/Subject_accessions.exercise.out
 create mode 100644 Lecture06/Top10.HSPs.exercise.out
 create mode 100644 Lecture06/al_leng_pcID.exercise.out
```

git lo

```
eea4f5e (HEAD -> master) Kept bash script output files from Lecture 06
fbe5cc0 (Lecture06/master) Added run_this_script_v1 from BPSM Lecture06
ce2c15a nematode BLASTDB stuff added
19cdd2d Added the example_people_data.tsv file from Lecture03
137c3db Decided to keep blastoutput2.out
08d379b Decided to keep myinputfile.tsv
c9b5b18 First attempt at parsing BLAST data
bbf825c My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeea I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```

git st | head

```
On branch master
Your branch is ahead of 'Lecture06/master' by 1 commit.
  (use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add/rm ..." to update what will be committed)
  (use "git restore ..." to discard changes in working directory)
        deleted:    ../Lecture04/CodeFiles.tar.gz
        modified:   run_this_script_v1
```

I cant recall my username or token... but were set up, so.....

set | grep myp

```
mypersonalGitHub_username=B123456-2121
mypersonalgittoken=ghp_mK...
```

git push Lecture06 master

```
Username for 'https://github.com': B123456-2121
Password for 'https://B123456-2121@github.com':
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 256 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (13/13), 11.57 KiB | 1.45 MiB/s, done.
Total 13 (delta 6), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (6/6), completed with 2 local objects.
To https://github.com/B123456-2121/BPSM_Lecture06.git
   fbe5cc0..eea4f5e  master -> master
```

git lo

```
eea4f5e (HEAD -> master, Lecture06/master) Kept bash script output files from Lecture 06
fbe5cc0 Added run_this_script_v1 from BPSM Lecture06
ce2c15a nematode BLASTDB stuff added
19cdd2d Added the example_people_data.tsv file from Lecture03
137c3db Decided to keep blastoutput2.out
08d379b Decided to keep myinputfile.tsv
c9b5b18 First attempt at parsing BLAST data
bbf825c My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeea I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```

Give feedback
Ask for help

Exercises: possible solutions using awk and/or grep rather than "pure" bash

 # List the subject ID (identity) for all HSPs

head -n 4 blastoutput2.out

```
# BLASTX 2.17.0+
# Query: NC_015119.1:1426-2962 Boreus elegans mitochondrion, complete genome
# Database: nem
# Fields: query acc.ver, subject acc.ver , % identity, alignment length, mismatches, gap o
```

From the "Fields" line, we can see that the second field is the HSP subject acc (accession/ID), so to get this field, we can just output the second field, but we also need to remove the comment lines.

grep -v "#" blastoutput2.out | cut -f2 | more

removes the comment lines, pipes the output to `cut`, which gets the second field, which is piped to `more` for ease of viewing

```
gi|225622197|ref|YP_002725710.1|
gi|288903410|ref|YP_003434131.1|
gi|326536058|ref|YP_004300493.1|
gi|188011119|ref|YP_001905892.1|
...
```

 # List the alignment length and percent ID for all HSPs

From the "Fields" line, we can see that the fourth field is the HSP alignment length, and the third field is the percent identity, so to get these fields, we can just output these two fields, but we also need to remove the comment lines.

grep -v "#" blastoutput2.out | cut -f4,3 | more

removes the comment lines, pipes the output to `cut`, which gets the fourth and third fields, which is piped to `more` for ease of viewing; note that `cut` **didn't** change the order of the output fields, even if we think it should have! How might we change the order, if we wanted to?

```
61.712  444
61.991  442
62.443  442
61.312  442
61.486  444
61.538  442
61.991  442
...
```

grep -v "#" blastoutput2.out | awk 'BEGIN{OFS="\t";}{print $4,$3;}'

Give feedback
Ask for help

removes the comment lines, pipes the output to `awk`, which prints out the fourth and third fields as tab-separated text **in that order**.

```
128     38.281
36      47.222
128     38.281
36      47.222
128     38.281
36      47.222
128     38.281
36      47.222
...
```

Show the HSPs with more than 20 mismatches

From the "Fields" line, we can see that the fifth field is the HSP mismatches, so to evaluate this field, we can use `awk`, but we also need to remove the comment lines.

grep -v "#" blastoutput2.out | awk '($5 > 20)' | more

removes the comment lines, pipes the output to `awk`, which checks (evaluates) to see if $5 is more than 20, and displays anything that meets the criterion, which is piped to `more` for ease of viewing.

```
NC_015119.1:1426-2962   gi|225622197|ref|YP_002725710.1|        61.712  444     169     1
NC_015119.1:1426-2962   gi|288903410|ref|YP_003434131.1|        61.991  442     167     1
NC_015119.1:1426-2962   gi|326536058|ref|YP_004300493.1|        62.443  442     165     1
NC_015119.1:1426-2962   gi|188011119|ref|YP_001905892.1|        61.312  442     170     1
NC_015119.1:1426-2962   gi|225622184|ref|YP_002725698.1|        61.486  444     170     1
...
```

This also works:

awk '($5 > 20 && $1 != "#")' blastoutput2.out | more

`awk` checks (evaluates) to see if $5 is more than 20 **AND** $1 isn't a #, and displays anything that meets these TWO criteria, which is piped to `more` for ease of viewing.

```
NC_015119.1:1426-2962   gi|225622197|ref|YP_002725710.1|        61.712  444     169     1
NC_015119.1:1426-2962   gi|288903410|ref|YP_003434131.1|        61.991  442     167     1
NC_015119.1:1426-2962   gi|326536058|ref|YP_004300493.1|        62.443  442     165     1
NC_015119.1:1426-2962   gi|188011119|ref|YP_001905892.1|        61.312  442     170     1
NC_015119.1:1426-2962   gi|225622184|ref|YP_002725698.1|        61.486  444     170     1
...
```

Show the HSPs shorter than 100 amino acids and with more than 20 mismatches

From the "Fields" line, we already know that the fifth field is the HSP mismatches, so this has to be more than 20. We are only interested in HSPs shorter than 100 amino acids, so we need the alignment length to be less than 100. Alignment

Give feedback
Ask for help

length is field four.

awk '($4 < 100 && $5 > 20 && $1 != "#")' blastoutput2.out | more

`awk` checks to see if $4 is less than 100 **AND** $5 is more than 20 **AND** $1 isn't a #, and displays anything that meets these THREE criteria, which is piped to `more` for ease of viewing.

```
NC_015119.1:1426-2962   gi|324524638|gb|ADY48446.1|     60.563  71      28      0       85
NC_015119.1:1426-2962   gi|401664211|dbj|BAM36458.1|    65.432  81      28      0       74
NC_015119.1:1426-2962   gi|402169024|emb|CCK73368.1|    47.059  51      27      0       41
NC_015119.1:1426-2962   gi|324575366|gb|ADY49962.1|     50.000  44      22      0       23
NC_015119.1:1426-2962   gi|402575841|gb|EJW69801.1|     41.096  73      43      0       69
NC_015119.1:1426-2962   gi|402583242|gb|EJW77186.1|     40.698  86      24      2       86
...
```

 # List the first 20 HSPs that have fewer than 20 mismatches

Number of mismatches are listed in the fifth field; we need those that are fewer than 20, but we only want the first 20, not all of them.

awk '($5 < 20 && $1 != "#")' blastoutput2.out | head -n20

`awk` checks to see if $5 is less than 20 **AND** $1 isn't a #, and displays anything that meets these TWO criteria, which is piped to `head`, which only shows the first 20.

```
NC_015119.1:1426-2962   gi|336287921|gb|AEI30249.1|     47.222  36      17      1       64
NC_015119.1:1426-2962   gi|336287923|gb|AEI30250.1|     47.222  36      17      1       64
NC_015119.1:1426-2962   gi|336287885|gb|AEI30231.1|     47.222  36      17      1       64
NC_015119.1:1426-2962   gi|336287889|gb|AEI30233.1|     47.222  36      17      1       64
NC_015119.1:1426-2962   gi|336287893|gb|AEI30235.1|     47.222  36      17      1       64
...
```

 # How many HSPs are shorter than 100 amino acids?

Alignment length is field four.

awk '($4 < 100 && $1 != "#")' blastoutput2.out | wc -l

`awk` checks to see if $4 is less than 100 **AND** $1 isn't a #, and displays anything that meets these TWO criteria, which is piped to `wc -l`, which counts the lines.

31

 # List the top ten highest (best) HSPs.

BLAST output is in decreasing score order, so the highest scoring HSPs are at the top of the output. These next lines of code are assuming that that is true....we haven't checked....

Give feedback
Ask for help

awk '($1 != "#")' blastoutput2.out | cut -f1,2,12 | head -n 10

`awk` checks to see if $1 isn't a #, gets fields 1, 2 and 12, which is piped to `head`, which displays the first 10 lines.

```
NC_015119.1:1426-2962	gi|225622197|ref|YP_002725710.1|	432
NC_015119.1:1426-2962	gi|288903410|ref|YP_003434131.1|	430
NC_015119.1:1426-2962	gi|326536058|ref|YP_004300493.1|	430
NC_015119.1:1426-2962	gi|188011119|ref|YP_001905892.1|	429
NC_015119.1:1426-2962	gi|225622184|ref|YP_002725698.1|	429
NC_015119.1:1426-2962	gi|171260186|ref|YP_001795390.1|	429
NC_015119.1:1426-2962	gi|288903312|ref|YP_003434040.1|	429
NC_015119.1:1426-2962	gi|49146527|ref|YP_026087.1|	428
NC_015119.1:1426-2962	gi|296315379|emb|CAQ56159.1|	428
NC_015119.1:1426-2962	gi|160694986|ref|YP_001552232.1|COX1_21467	428
```

This also works, but maybe not always giving the same answer...

grep -v "#" blastoutput2.out | cut -f1,2,12 | head -n 10

`grep` suppresses lines with a # anywhere in the line, the rest is piped to `head`, which displays the first 10 lines. It's better/safer to use the more specific version! 😀

 # List the start positions of all matches where the HSP name includes the letters string "AEI".

From the "Fields" line, we can see that there are two start fields: q. start (query start, field 7) and s. start (subject start, field 9).

We weren't told which one is the one we are after, so we will make an arbitrary decision to go with q. start (query start, field 7). When reporting the results, we should just say which one is being used: it's not your fault that the question was ambiguous (it was mine, deliberately!)...

HSP name is the subject ID, so that is field 2. If we are looking for a string of letters, this pattern matching could be achieved with, for example, grep, but remember that grep searches need to be carefully focussed!

cut -f2,7 blastoutput2.out| grep "AEI" | cut -f2

`cut` extracts fields 2 and 7, piped to `grep` to select the lines with AEI in them, piped to `cut` to retain only the second field of the pipe contents, which in this case is the start position.

```
238
641
238
641
238
641
238
...
```

How many subject sequences have more than one HSP?

The text "how many" should immediately (hopefully) suggest `uniq -c` to you! We are not asked about how good the HSPs are, or what they are, just how many there are. The subject IDs are what we after here, so we just need a sorted list of this field, field 2.

```
cut -f2 blastoutput2.out | \
 sort | \
 uniq -c | \
 awk '($1 > 1)' | \
 wc -l
```

`cut` extracts field 2, which is then piped to `sort`, then piped to `uniq` for counting, then `awk` to evaluate the output and exclude single instances, then piped to `wc -l` to count how many lines met all criteria.

```
23
```

The word "pipe" is still being used frequently: as you will recall, this is essentially a pipe(line) of actions, each of which depends on output from all previous actions.

The code was also written in a more structured form, using continuation: makes it less error-prone and easier to read than this:

```
cut -f2 blastoutput2.out | sort | uniq -c | awk '($1 > 1)' | wc -l
```

What percentage of each HSP is made up of mismatches?

The numbers we need are the number of mismatches (field 5), and the length of the alignment (field 4).

```
grep -v "#" blastoutput2.out | \
  awk '{
HSP_mm_pc = ($5/$4) * 100 ;
print HSP_mm_pc;
       }' |  more
```

What is it doing? For each line of the blastoutput2.out BLAST output:

1. `grep -v` suppresses lines with a "#"
2. pipes to `awk`
3. which divides field 5 by field 4

4. then converts this proportion to a percentage by multiplying by 100
5. then "saves" it as an internal variable called HSP_mm_pc
6. then outputs (prints) the value of this variable
7. which is finally piped to `more` for ease of viewing

```
38.0631
37.7828
37.3303
38.4615
38.2883
38.2353
...
```

The lines above are also a small pipeline that is essentially being executed as a single line of code, just with nicer visual formatting.

Remember how we need the "line breaks" for the Linux (bash) command line, but not for `awk`.

As a reminder again, some languages such as Python **do** have strict code formatting requirements...!

```
# Time to do our usual git repository checks
git st
```

```
On branch master
Your branch is up-to-date with 'Lecture06/master'.

Changes not staged for commit:
  (use "git add/rm ..." to update what will be committed)
  (use "git restore ..." to discard changes in working directory)
        deleted:    ../Lecture04/CodeFiles.tar.gz
        modified:   run_this_script_v1

Untracked files:
  (use "git add ..." to include in what will be committed)
        ../BN.out
        ../BN.sh
        ../Lecture04/Als_ICA/
        ../Lecture04/all_the_things_I_did
        ../Lecture04/outs/
        ../Lecture05/Country.Mozambique.details

no changes added to commit (use "git add" and/or "git commit -a")
```

What should you do next?!

Give feedback
Ask for help

Have your say...

The "Postbox of Truth": is your chance to give anonymous "in course" feedback/comments. Your comments are completely anonymous and confidential, so say what you think, irrespective of whether the comments are positive or otherwise! You can use this multiple times too: I use it throughout the course to enable you to give context-relevant feedback/make suggestions.

Just click the image!